CODEMENTOR AI: A MULTI-AGENT AI FRAMEWORK FOR AUTOMATED PROGRAMMING ASSESSMENT AND PERSONALIZED EDUCATIONAL FEEDBACK

VIJIL RAJ
TCR25MCA-2057

Mini Project

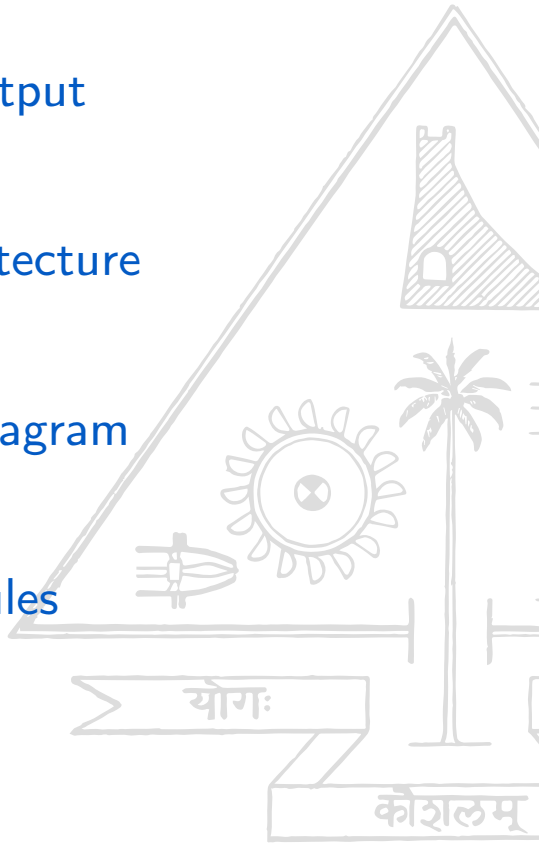


Guided By
Sreejith V P

Department of Computer Applications
Government Engineering College, Thrissur, Kerala

Outline

1. Introduction
2. Literature Review
3. Existing System
4. Proposed System
5. Requirements Specification
6. Input and Output
7. System Architecture
8. Data Flow Diagram
9. System Modules
10. Conclusion





Introduction

- Programming laboratories occupy a central place in CS/ MCA education, translating theory into working software.
- Faculty must evaluate large volumes of submissions across data structures, algorithms, and language-specific constructs (C, Java, Python), leaving little time to read every line of code.
- As a result, assessment often reduces to a binary check of output correctness rather than an evaluation of the reasoning behind it.



Introduction

- Students who reach a correct answer through very different levels of understanding end up receiving the same grade.
- There is no reliable mechanism to check whether a submission reflects genuine understanding or was produced/ copied with minimal student effort.
- Faculty therefore need an automated way to assess not just *whether* a program runs, but *how well* the student understands the code they have submitted.



Introduction

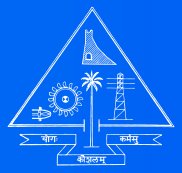
Objectives

- Develop a multi-agent AI framework for automated programming assessment and personalized educational feedback.
- Combine deterministic code analysis (*AST generation, static analysis, complexity measurement*) with specialized, code-tuned LLM agents.
- Reduce faculty workload through automation of routine assessment tasks.
- Provide students with continuous, concept-level guidance beyond simple pass/ fail grading.

Literature Review

Zhao, J. et al., "CodeEdu: A Multi-Agent Collaborative Platform for Personalized Coding Education, " *arXiv preprint arXiv:2507.13814*, 2025.

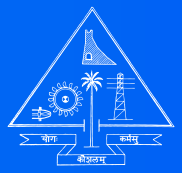
- Introduced a *multi-agent platform* with agents for *task planning, personalized material generation, real-time Q&A, step-by-step tutoring, debugging, and learning reports.*
- Automated evaluations showed measurable improvement in coding performance over a single-agent baseline.
- **Observation:** Validates multi-agent decomposition for coding education, but the agents support students *while* they code – none formally score a completed submission, the gap CodeMentor AI targets.



Literature Review

Kazemitabaar, M. et al., "CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs," *Proc. CHI Conference on Human Factors in Computing Systems*, 2024.

- Deployed a pedagogical assistant that withholds direct solutions while answering conceptual questions and annotating incorrect code.
- Semester-long study (700 students) via logged interactions, surveys, and educator interviews.
- **Observation:** Confirms LLM assistants are effective for real-time guidance at classroom scale, but stop short of assessing finished work – reinforcing the need for a dedicated post-submission evaluation layer.



Literature Review

Yousef, M. et al., "BeGrading: Large Language Models for Enhanced Feedback in Programming Education, " *Neural Computing and Applications*, 2025.

- Fine-tuned an LLM on historical assignments and instructor-assigned grades, augmented with additional grading scenarios.
- Achieved smaller deviation from reference grades than an off-the-shelf coding model; human TA grading itself showed notable inconsistency.
- **Observation:** LLM scoring approximates but does not match instructor judgment – the direct motivation for CodeMentor AI's confidence-calibrated, faculty-override design rather than fully autonomous grading.

Literature Review

Huang, D. et al., "AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation," *arXiv preprint arXiv:2312.13010*, 2023.

- Three cooperating agents: *programmer, independent test designer, and test executor* that relays failures back to the programmer.
- Separating code generation from test generation avoided bias from a single model writing both.
- **Observation:** Supports CodeMentor AI's choice to keep mastery estimation and plagiarism detection as separately specialized agents rather than one monolithic evaluator, even while sharing a common backbone.

Literature Review

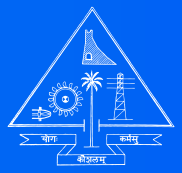
Song, J. et al., "Improving Code Summarization With Tree Transformer Enhanced by Position-Related Syntax Complement," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 9, 2024.

- Syntax-tree-aware attention over AST structure improves accuracy over models trained on flattened token sequences.
- Position-related syntax information, folded into the tree representation, captures structural relationships plain text tokenization discards.
- **Observation:** Directly motivates CodeMentor AI's core design choice – *deriving trajectory features from AST structure* rather than raw code text – as the shared backbone for mastery estimation and authorship-drift detection.

Literature Review

Author	Approach	Brief Observation
Zhao et al. (2025)	Multi-Agent Tutoring	Coordinated agents aid coding, but don't grade a finished submission.
Kazemitabaar et al. (2024)	Classroom LLM Assistant	Guides students during coding; no post-submission assessment.
Yousef et al. (2025)	Fine-Tuned LLM Grading	LLM scoring approximates instructor grades – best as a complement, not a replacement.
Huang et al. (2023)	Multi-Agent Code Gen + Test	Specialized, separated agents outperform a single monolithic agent.
Song et al. (2024)	AST-Aware Tree Transformer	Structural (AST) representations outperform flattened token sequences.

Table 1: Summary Of Reviewed Literature

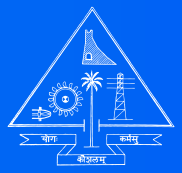


Existing System

- **Manual Evaluation:** Judgement-based, doesn't scale – forces superficial, output-only checks as class sizes grow.
- **Simple Automated Grading:** Pass/ fail output-matching; cannot detect poor practices, inefficient algorithms, or conceptual gaps.
- **General-Purpose AI Assistants:** Explain/ fix code on request, but aren't integrated into assessment – no formal evaluation of submitted work.
- **Viva Selection:** No tool generates personalized viva questions to verify genuine understanding.
- **Plagiarism Checks:** Text/ token-based plagiarism checks are defeated by renaming or reordering, with no explainable match – and faculty get only a raw score, not class-level analytics.

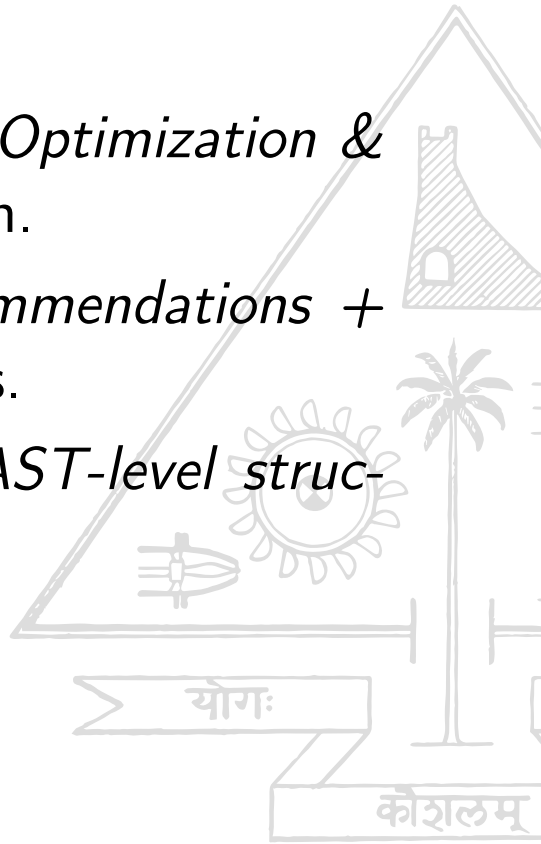
Proposed System

- Multi-agent AI framework for automated programming assessment and personalized feedback (C, Java, Python).
- **Language Adapter Layer:** AST generation, static analysis, complexity measurement, sandboxed execution, test evaluation.
- **Question Setup (faculty-triggered, once per assignment):** A Test Case Agent and a Viva Question Agent generate a candidate test suite and viva question bank for the question; faculty reviews and approves both before the assignment is published to students.



Proposed System

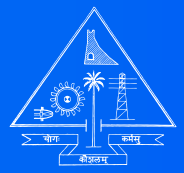
- **Orchestration Engine:** *Assessment, Mentor, Optimization & Viva Agents* share one structured representation.
- **Faculty Intelligence Module:** *Grading recommendations + class-level analytics* on common misconceptions.
- **Academic Integrity Module:** *Explainable, AST-level structural similarity* – not opaque percentages.



Requirements Specification

Hardware Requirements

- **Processor**: Intel® Core™ i5 (8th Gen or higher)/ AMD Ryzen™ 5 or higher, at least 4 physical cores
- **Memory**: Minimum 16 GB RAM
- **Storage**: 100–150 GB free disk space
- **Network**: Stable internet connection for API-based LLM inference (no dedicated GPU required)



Requirements Specification

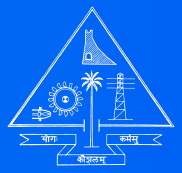
Software Development Requirements

- **Operating System**: Microsoft Windows® 10/ 11/ Ubuntu® 22.04 LTS/ macOS®
- **Back End**: Python 3.10 or higher, FastAPI
- **Front End**: React™ (JavaScript ES6+)
- **Database**: PostgreSQL®
- **Containerization**: Docker®, Docker Compose (sandboxed execution)



Requirements Specification

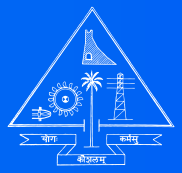
- **IDE Used: Visual Studio Code**
- **Version Control**: Git
- **Development Tools and Frameworks**:
  - Language parsers – `ast` (Python), `javalang` (Java), `pycparser` + `cppcheck` (C)
  - Radon – cyclomatic complexity & maintainability metrics
  - Cloud LLM inference API (Together AI™/ Fireworks AI™/ Groq™) serving DeepSeek-Coder, Qwen2.5-Coder, Code Llama™
  - Postman – REST API testing



Requirements Specification

Functional Requirements

- **User Roles & Interfaces:** Student Portal for *Submission and feedback*; Faculty Dashboard for *Question Setup, Grading, Integrity, and Analytics*.
- **Question Setup & Assignment Creation:** Faculty defines a question in the Faculty Dashboard; this triggers a *Test Case Agent and a Viva Question Agent* to generate a candidate test suite and viva question bank, which faculty *review, edit, and approve* before the assignment is published.
- **Source Code Submission:** Accepts files against a published assignment; validates before queuing.

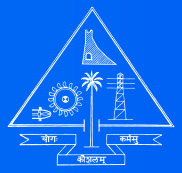


Requirements Specification

- **Language Adapter & Parsing:** Generates an AST per language; syntax errors returned as immediate diagnostics.
- **Static Analysis, Complexity & Secure Execution:** Detects coding-standard violations, computes complexity metrics, executes in an isolated, network-less sandbox against the faculty-approved test suite.
- **Faculty Intelligence Module:** Consolidated grading recommendation + class-level analytics.
- **Academic Integrity Module:** Structural similarity comparison with an explainable match report.

Requirements Specification

- **Multi-Agent AI Orchestration** (run concurrently):
  - **Assessment Agent** – Correctness against the faculty- approved test suite, coding standards, efficiency, with justification.
  - **Mentor Agent** – Concise, student-friendly conceptual explanation.
  - **Optimization Agent** – Refactoring and performance suggestions tied to specific lines.
  - **Viva Agent** – Selects and personalizes questions from the faculty-approved bank based on the concepts and constructs present in the student's submission.
- **Outputs:** Assessment Report, Mentor Explanation, Optimization Suggestions, Viva Question Set, Integrity Report.



Requirements Specification

Non-Functional Requirements

- **Scalability:** Handles class-wide concurrent submissions via independently scalable services.
- **Usability:** Self-explanatory Student Portal and Faculty Dashboard, no specialized training needed.
- **Reliability:** Gracefully handles malformed submissions and execution failures without disrupting the pipeline.
- **Maintainability:** Modular, decoupled agents/ adapters – easy to extend with a new language or agent.
- **Security & Data Privacy:** Isolated, network-less sandbox for untrusted code; secure storage; auditable open-source LLMs.

Input and Output

- **Faculty Input (Question Setup, once per assignment):** Question title, description, and constraints entered in the Faculty Dashboard.
- **Faculty Output (Question Setup):** Agent-generated candidate test suite and viva question bank, presented for faculty review, edit, and approval before publishing.
- **Student Input:** A single source code file.
- **Input Format:** Source code files written in supported programming languages (C, Java, Python).



Input and Output

- **Student-Facing Output**
  - Automatically generated, per-submission assessment report.
  - Mentor Explanation for concept-level understanding.
  - Optimization Suggestions for code improvement.
  - Personalized Viva Question Set drawn from the approved bank.
  - Integrity Report, where applicable.
- **Output Format:** *Structured, agent-generated feedback delivered through the Student Portal and Faculty Dashboard*, alongside consolidated grading recommendations and class-level analytics for faculty.

System Architecture

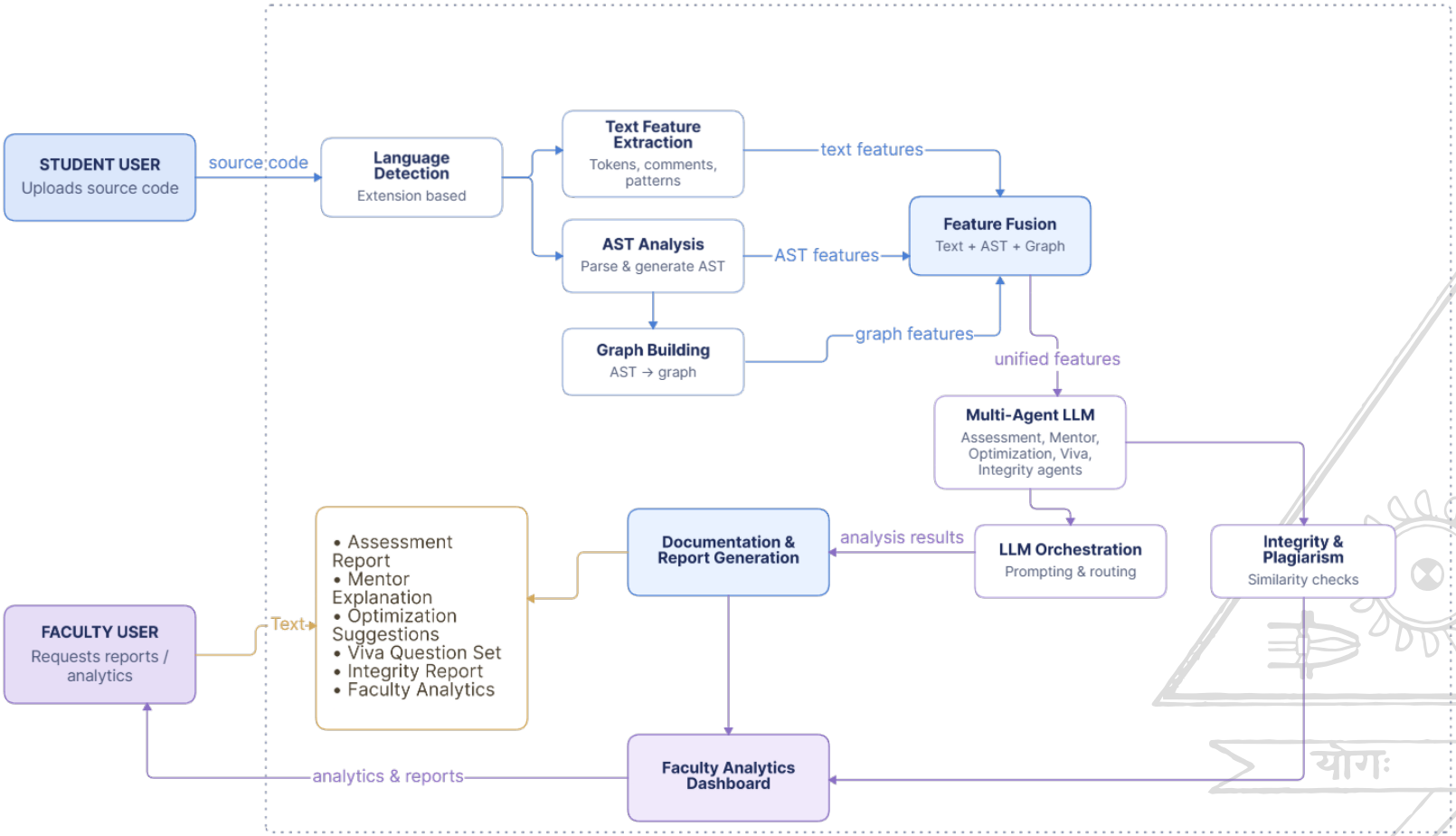


Figure 1: CodeMentor AI System Architecture

Data Flow Diagrams

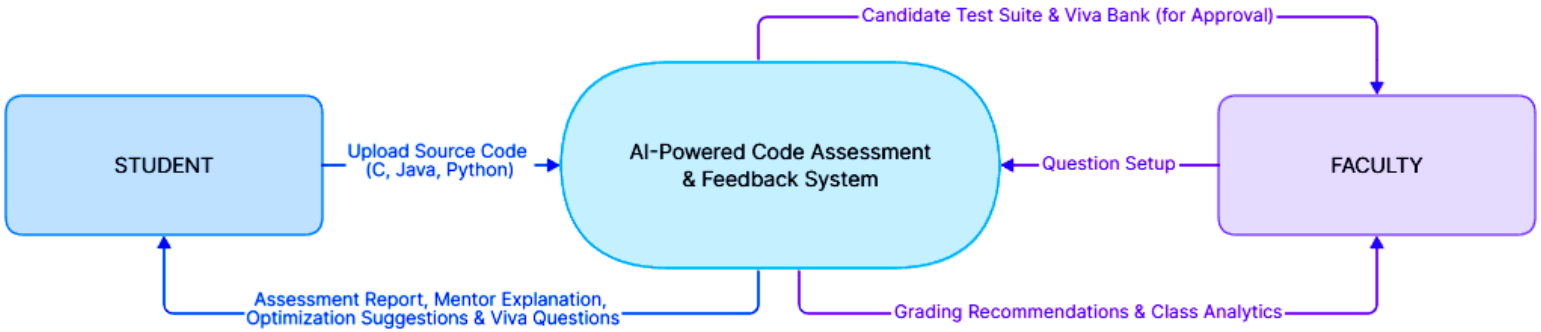


Figure 2: Level 0 DFD

Data Flow Diagrams

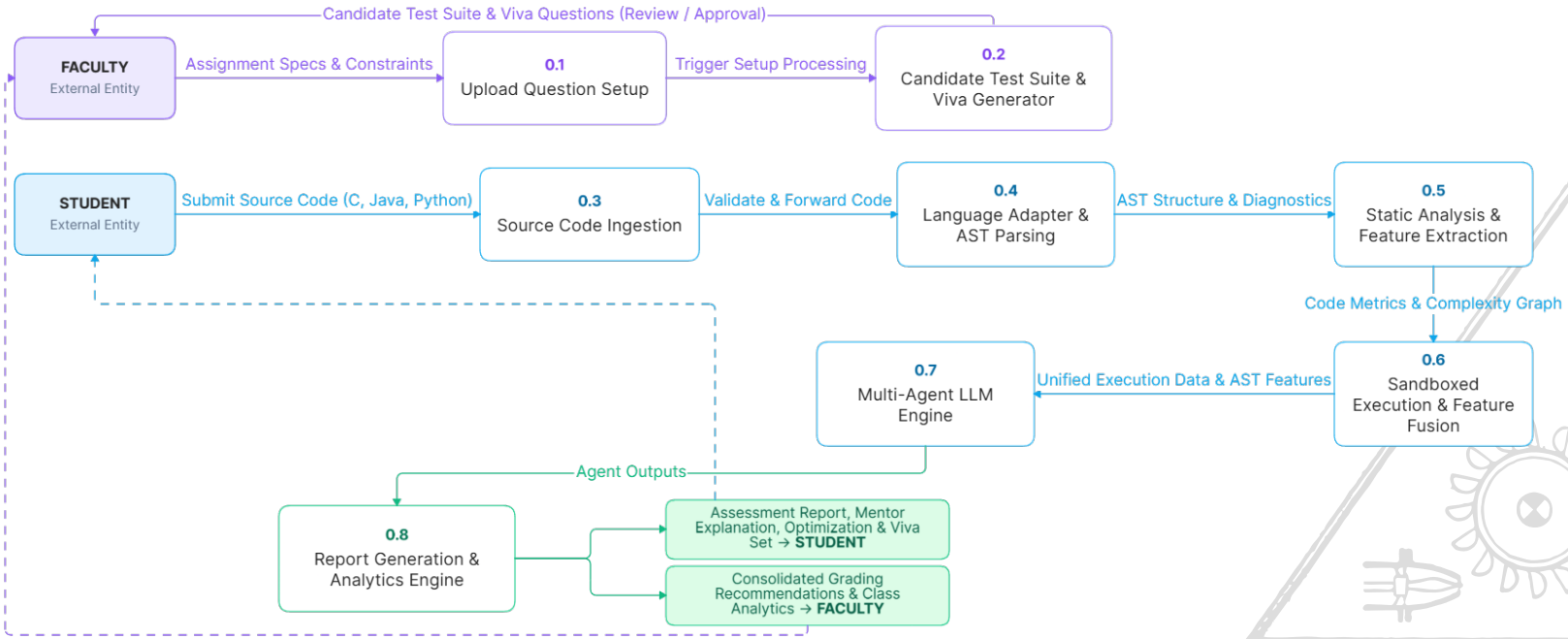


Figure 3: Level 1 DFD



System Modules

- **Student Portal Module:** Provides students with an interface to submit source code against published assignments and view assessment reports, mentor explanations, optimization suggestions, and viva questions.
- **Faculty Dashboard Module:** Provides faculty with an interface for question setup, reviewing and approving generated test suites and viva banks, grading, academic-integrity review, and class-level analytics.
- **Question Setup Module:** Runs the Test Case Agent and Viva Question Agent once per assignment to generate a candidate test suite and viva question bank for faculty review and approval before publishing.



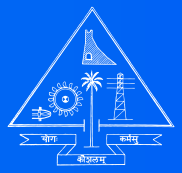
System Modules

- **Language Adapter Layer Module:** Prepares each submission for analysis and evaluation through three sub-components:
  - *AST Generator* — Parses the submission and generates an Abstract Syntax Tree for each supported language (C, Java, Python), returning syntax errors as immediate diagnostics.
  - *Static Analyzer & Radon* — Detects coding-standard violations and computes cyclomatic complexity and maintainability metrics from the parsed code.
  - *Sandboxed Execution* — Executes the submission in an isolated, network-less sandbox against the faculty-approved test suite and evaluates the results.



System Modules

- **Orchestration Engine (Multi-Agent Module):** Coordinates the Assessment, Mentor, Optimization, and Viva Agents over a shared structured representation of the submission, running them concurrently.
- **Faculty Intelligence Module:** Consolidates agent outputs into a grading recommendation and generates class-level analytics on common misconceptions.
- **Academic Integrity Module:** Performs explainable, AST-level structural similarity comparison between submissions and produces a match report.



Conclusion

- CodeMentor AI combines deterministic code analysis (*AST,static analysis, complexity*) with a multi-agent, open-source-LLM architecture for programming assessment.
- Unlike existing AI coding assistants that emphasize code generation, it focuses on *assessment, personalized feedback, faculty decision support, academic integrity, and learning analytics* within one platform.
- Aims to reduce *instructor workload, improve consistency of assessment, promote academic integrity, and give students continuous, concept-level guidance.*
- Grounded in open-source LLMs accessed via inference APIs, the system is designed to be *transparent, explainable, and affordable* at institutional scale.



References

[1] J. Zhao, P. Gao, J. Cao, Z. Wen, C. Chen, J. Yin, R. Yang and B. Yuan, "CodeEdu: A Multi-Agent Collaborative Platform for Personalized Coding Education," *arXiv preprint arXiv:2507.13814*, 2025.

[2] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig and T. Grossman, "CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant that Balances Student and Educator Needs," in *Proc. 2024 CHI Conference on Human Factors in Computing Systems (CHI '24)*, 2024, pp. 1–20.



References

[3] M. Yousef, K. Mohamed, W. Medhat, E. H. Mohamed, G. Khoriba and T. Arafa, "BeGrading: Large Language Models for Enhanced Feedback in Programming Education, " *Neural Computing and Applications*, vol. 37, pp. 1027–1040, 2025.

[4] D. Huang, Q. Bu, J. M. Zhang, M. Luck and H. Cui, "Agent-Coder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation, " *arXiv preprint arXiv:2312.13010*, 2023.

[5] J. Song, Z. Zhang, Z. Tang, S. Feng and Y. Gu, "Improving Code Summarization With Tree Transformer Enhanced by Position-Related Syntax Complement, " *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 9, Sept. 2024.



Thank You